

CST1510 – Programming for Data Communication and Networks

Assessment: Coursework 2 (CW2)

Student: M01083909

GitHub: https://github.com/PedroOshiro453/CW_CST1510_M01083909

Reflection on Development Challenges

Introduction

The development of the Multi-Domain Intelligence Platform for CST1510 Coursework 2 was a challenging and valuable learning experience. Although the final application works as expected the process of building it was not always smooth. Many of the main difficulties did not come from writing the code itself, but from setting up the development environment, connecting the data correctly, and understanding why the application did not always behave as expected.

This reflection discusses the main challenges faced during the project. These include problems with committing the project from Visual Studio Code to GitHub, difficulties caused by Conda and virtual environments, issues when connecting data files and displaying information correctly, and situations where the problem was not the code, but cached data and application state. These challenges helped demonstrate that programming is not only about writing correct code, but also about managing tools, environments, and data.

Problems with GitHub and Visual Studio Code

One of the biggest challenges during the project was committing the work from Visual Studio Code to GitHub. At first, this seemed like a simple task, but in practice it caused many problems. Sometimes changes appeared in the editor but did not appear in the GitHub repository after committing. This created confusion and uncertainty about whether the project was being saved correctly.

The main reason for this problem was that Visual Studio Code was not always using the same environment where the required tools and extensions were installed. In some cases, Git features worked as expected, but in other cases, the editor behaved differently. This made it difficult to trust that commits were being done correctly.

Another issue was understanding whether GitHub problems were caused by mistakes in Git commands or by problems with the editor setup. Over time, it

became clear that the issue was not with Git itself, but with how Visual Studio Code was connected to the active environment.

This experience showed how important it is to:

- Confirm which environment is active in the editor
- Make sure all tools are installed in the same place
- Avoid switching environments unnecessarily

Once the environment was fixed and kept consistent, committing to GitHub became much more reliable.

Difficulties with Conda and Virtual Environments

Managing Conda and virtual environments was another major challenge. During development, more than one environment was available, and it was not always clear which one was being used at any given time. Some libraries worked in one environment but were missing in another, even though the code had not changed.

This caused situations where the application worked correctly in the terminal but failed when run from Visual Studio Code, or the opposite. These errors were frustrating because they looked like code problems but were actually caused by missing libraries in the active environment.

Conda added extra complexity because some extensions and tools were only available in specific environments. When Visual Studio Code switched environments automatically, those tools disappeared, which made debugging harder.

To solve this issue, a single environment was chosen and used consistently. All required libraries were installed there, and the environment was activated before running or committing the project. This reduced confusion and made the application behaviour more predictable.

This challenge helped highlight that environment management is just as important as writing correct code.

Challenges Connecting Data Files

Another important difficulty was connecting the data files and displaying the expected information in the dashboard. Even though the CSV files were present in the project folder, the dashboard sometimes showed incomplete data or did not display the expected results.

One problem was related to file paths. In some cases, the application tried to load files based on the working directory rather than the actual location of the script. This caused the program to read the wrong file or fail silently.

Another issue was the structure of the data itself. Some columns contained values that were difficult to work with, especially the timestamp column, which only stored time values such as “00:00.0”. Because there was no real date information, the application could not easily show a meaningful timeline.

At first, it seemed that the charts were broken. However, after careful testing, it became clear that the problem was the data format rather than the visualisation code. The solution was to validate the data structure and create a safe way to display trends even when the original data was limited.

This process showed how important it is to understand the data before trying to display it.

When the Code Was Not the Problem

One of the most important lessons from this project was learning that sometimes the code is correct, but the application still does not show the expected result. This happened several times during development, especially when changes to the layout or charts did not appear after updating the code.

The cause of this problem was cached data and saved application state. The dashboard reused previous results instead of recalculating everything. This made it look like the code changes were not working.

At first, this led to unnecessary changes and confusion. Only after clearing the cache and restarting the application did the updates appear correctly. Once this was understood, clearing the cache became a standard step during debugging.

This experience helped reinforce an important idea: not every error comes from logic or syntax. Sometimes the environment or stored state is responsible.

Learning to Debug More Effectively

As the project progressed, debugging became more structured and efficient. Instead of immediately rewriting code, the focus shifted to checking:

- Whether the correct environment was active
- Whether the correct data files were being loaded
- Whether cached information needed to be cleared

This approach saved time and reduced frustration. It also helped build confidence in understanding how the application actually runs behind the scenes.

Learning to slow down and analyse the full context of a problem was one of the most valuable skills developed during this coursework.

Broader Lessons from the Project

This project demonstrated several important lessons that go beyond this specific assignment:

- Development tools can cause as many problems as the code itself
- Clear organisation of files and environments reduces errors
- Data quality directly affects how useful visualisations are
- Debugging requires patience and systematic thinking

These lessons are relevant not only for academic projects, but also for real-world development work.

Conclusion

The development of the Multi-Domain Intelligence Platform was a challenging but rewarding experience. While the final application meets the project requirements, the most valuable learning came from dealing with unexpected problems related to environments, data handling, and application behaviour.

Issues with GitHub commits, Conda environments, CSV data, and caching showed that successful programming depends on more than just writing correct code. It requires understanding how tools work together and how data flows through an application.

By overcoming these challenges, this project helped improve both technical skills and problem-solving ability. It also provided a realistic view of the kind of difficulties that developers often face in practice, making it a valuable learning experience.